

Verification Continuum™

Verdi® Python-Based NPI UPF Model

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	10
Synopsys Statement on Inclusivity and Diversity	11

1. Introduction to Python Based NPI	12
Packages and Modules	12
Packages	12
Modules	12
User Interface and Use Flow	13
Environment and Library Setting	13

2. Module npisys	15
Overview	15
L0 APIs	15
npisys.init(<i>pyArgvList</i>)	15
npisys.load_design(<i>pyArgvList</i>)	15
npisys.end()	16

3. Python Based NPI UPF Model	17
Abstract	17
Quick Start	17
Quick Start	18
Environment and library setting	18
Example	18
Execution	21
Result	21
Properties and Enum	22
Classes	23
Enums	24
ClassType	24
Property strings and integers	24
Class Sets	29
BaseNamedSet	29

Contents

NetworkClassSet	30
PowerDomainHdl	30
parent()	31
creation_scope()	32
supply_set_handles()	32
retention_strategies()	32
isolation_strategies()	32
repeater_strategies()	33
pd_states()	33
type(<i>get_enum=False</i>)	33
name()	33
full_name()	34
file()	34
line()	34
RetentionStrategyHdl	35
parent()	36
creation_scope()	37
effective_extents()	37
save_condition()	37
restore_condition()	37
retention_condition()	38
save_signal()	38
restore_signal()	38
logic_refs()	38
type(<i>get_enum=False</i>)	39
name()	39
full_name()	39
file()	39
line()	40
is_no_retention()	40
is_use_retention_as_primary()	40
IsolationStrategyHdl	41
parent()	42
creation_scope()	42
source_filter()	43
sink_filter()	43
supply_set_handles()	43
logic_refs()	43
type(<i>get_enum=False</i>)	44
name()	44
full_name()	44
file()	45
line()	45
location(<i>get_enum=False</i>)	45

Contents

applies_to(<i>get_enum=False</i>).....	45
is_use_equivalence().....	46
is_diff_supply_only().....	46
is_no_isolation().....	46
clamp_value(<i>get_enum=False</i>).....	46
RepeaterStrategyHdl	47
parent().....	48
creation_scope().....	48
source_filter().....	48
sink_filter().....	49
type(<i>get_enum=False</i>).....	49
name().....	49
full_name().....	49
file().....	50
line().....	50
location(<i>get_enum=False</i>).....	50
applies_to(<i>get_enum=False</i>).....	50
SupplySetHdl	51
parent().....	52
creation_scope().....	52
functions().....	52
ss_states().....	53
type(<i>get_enum=False</i>).....	53
name().....	53
full_name().....	54
file().....	54
line().....	54
NamedRefHdl	55
parent().....	55
creation_scope().....	56
ref_object().....	56
type(<i>get_enum=False</i>).....	56
name().....	57
full_name().....	57
file().....	57
line().....	57
ref_kind(<i>get_enum=False</i>).....	58
SupplyNetHdl	58
parent().....	59
creation_scope().....	59
type(<i>get_enum=False</i>).....	59
name().....	59
full_name().....	60
file().....	60

Contents

line().....	60
SupplyPortHdl	61
parent().....	62
creation_scope().....	62
hiconn().....	62
sp_states().....	63
type(<i>get_enum=False</i>).....	63
name().....	63
full_name().....	63
file().....	64
line().....	64
port_dir(<i>get_enum=False</i>).....	64
LogicNetHdl	65
parent().....	65
creation_scope().....	66
type(<i>get_enum=False</i>).....	66
name().....	66
full_name().....	66
file().....	67
line().....	67
LogicPortHdl	67
parent().....	68
creation_scope().....	68
hiconn().....	69
type(<i>get_enum=False</i>).....	69
name().....	69
full_name().....	69
file().....	70
line().....	70
PowerSwitchHdl	71
parent().....	72
creation_scope().....	72
output_supply_port().....	72
control_ports().....	73
ack_ports().....	73
sw_states().....	73
type(<i>get_enum=False</i>).....	73
name().....	74
full_name().....	74
file().....	74
line().....	75
AckPortHdl	75
parent().....	76

Contents

creation_scope()	76
hiconn()	76
type(<i>get_enum=False</i>)	77
name()	77
full_name()	77
file()	77
line()	78
port_dir(<i>get_enum=False</i>)	78
PowerStateHdl	79
parent()	80
creation_scope()	80
logic_expr()	80
supply_expr()	80
type(<i>get_enum=False</i>)	81
name()	81
full_name()	81
file()	82
line()	82
is_illegal()	82
simstate(<i>get_enum=False</i>)	82
PowerSwitchStateHdl	83
parent()	84
creation_scope()	84
switch_expr()	84
input_supply_port()	85
type(<i>get_enum=False</i>)	85
name()	85
full_name()	85
file()	86
line()	86
is_illegal()	86
switch_output_state(<i>get_enum=False</i>)	86
SupplyPortStateHdl	87
parent()	88
creation_scope()	88
type(<i>get_enum=False</i>)	88
name()	88
full_name()	89
file()	89
line()	89
supply_state(<i>get_enum=False</i>)	90
volt_min()	90
volt_nom()	90
volt_max()	90

Contents

volt_kind(<i>get_enum=False</i>)	91
PstStateHdl	91
parent()	92
creation_scope()	92
type(<i>get_enum=False</i>)	92
name()	93
full_name()	93
file()	93
line()	93
PowerStateTableHdl	94
parent()	95
creation_scope()	95
pst_states()	95
pst_header()	95
type(<i>get_enum=False</i>)	96
name()	96
full_name()	96
file()	96
line()	97
ExtentHdl	97
object()	98
next_extent()	98
type(<i>get_enum=False</i>)	98
SignalSenseHdl	99
control_signal()	99
type(<i>get_enum=False</i>)	99
signal_sensitivity(<i>get_enum=False</i>)	100
ExpressionHdl	100
expr_operands()	100
type(<i>get_enum=False</i>)	101
expr_string()	101
PowerScopeHdl	101
parent()	102
type(<i>get_enum=False</i>)	102
name()	102
full_name()	103
L0 APIs	103
Demo Case for Documents	103
Design	103
Public Function	106
upf.handle_by_name(<i>pathname, relativeTo</i>)	106
upf.get_top_scope()	106

Contents

upf.get_power_domains().....106

upf.get_power_switches().....107

upf.get_power_state_tables()..... 107

Preface

The Python Based NPI Netlist Model User Guide provides convenient APIs to access netlist.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetsynopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

- You can call for support at:

<https://www.synopsys.com/support/global-support-centers.html>

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to Python Based NPI

Python-Based NPI APIs support seven models. They are as follows:

- Language
- Netlist
- Waveform
- Text
- Coverage
- Waveform Writer
- UPF

Each model have their own APIs to let you be able to traverse data objects and obtain objects' properties.

In this guide, the environment setting for using **Python-Based NPI APIs for UPF** is demonstrated.

Packages and Modules

Packages

The Python-based NPI package name is “pynpi”, and it is placed at `$VERDI_HOME/share/NPI/python`.

Modules

There are eight modules inside the “pynpi” package: npisys, lang, netlist, text, cov, waveform, waveformw, and upf. The first module, npisys, is the system model for initialization, loading design and exit. The other modules represent language model, netlist model, text model, coverage model, wave model, waveform writer model, and UPF model respectively

User Interface and Use Flow

This chapter describes the user interface and use flow for Python-Based NPI APIs.

Environment and Library Setting

The python library setting flow of using Python-Based NPI APIs contains four parts:

1. Check your Python's version:

Python-Based NPI APIs need the Python version greater than 3.6.0.

2. Environment setting for “VERDI_HOME” is required for Python-based NPI. Ensure that you set it up before running program.
3. Add python library path into your python code before loading Python-Based NPI by the following commands:

```
rel_lib_path = os.environ['VERDI_HOME'] + '/share/NPI/python'
sys.path.append(os.path.abspath(rel_lib_path))
```

4. Import module “npisys” for using the function of NPI initialization and exit from pynpi package.

```
from pynpi import npisys
```

5. Import the module you need from pynpi package. For example, if you want to use UPF model, you can import the module as follows:

```
from pynpi import upf
```

6. Note that initialization function `npisys.init()` must be called before writing your code by using any other modules. Besides, `npisys.end()` should be called after finishing your code. Following is a simple example to demonstrate how to use UPF model by Python-Based NPI APIs.

Python program to use NPI UPF model: (demo.py)

```
#!/usr/bin/env python3

import sys, os
rel_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(rel_lib_path))
from pynpi import npisys
from pynpi import upf

# Initialize NPI
if not npisys.init(sys.argv):
    print("Error: Fail to initialize NPI")
    assert 0
```

```
# Beginning of your code here -----  
#  
# Example code can be found in later chapters  
#  
# End of your code -----  
  
# End NPI  
npisys.end()
```

C shell script to setup environment and execute Python program on 64-bit machine:
(run_demo)

```
#!/bin/csh -f  
# Setup your $VERDI_HOME here  
setenv VERDI_HOME [YOUR_VERDI_HOME_PATH]  
# run the python program  
# - Input arguments depend on your program design  
# - If loading design is required, you can pass the options like  
./demo.py -dbdir simv.daidir
```

To run the example, put the above files in the same directory and execute the C shell
script run_demo.

./run_demo

2

Module npisys

This chapter contains the following topics:

- [Overview](#)
- [L0 APIs](#)

Overview

Module npisys is for setting Python-based NPI. You must call `npisys.init()` before using any other NPI modules and call `npisys.end()` after using any other NPI modules.

L0 APIs

Following are the public L0 APIs for system module:

npisys.init(*pyArgvList*)

System initialization for Python-Based NPI.

Parameters: **pyArgvList** (str list) – input argument list, for example, `sys.argv`

Returns: Return 1 if successful. Otherwise, return 0.

Return type: int

Example:

```
>>>npisys.init(sys.argv)
```

npisys.load_design(*pyArgvList*)

Load design for Python-Based NPI.

Parameters: **pyArgvList** (str list) – input argument list, for example, `sys.argv`

Returns: Return 1 if successful. Otherwise, return 0.

Return type: int

Example:

```
>>>npisys.load_design(sys.argv)
```

npisys.end()

Clean NPI-related settings and data.

Parameters: none

Returns: Return 1 if successful. Otherwise, return 0.

Return type: int

Example:

```
>>>npisys.end()
```


3

Python Based NPI UPF Model

Python-Based NPI Upf Model is used to provide APIs for external users getting their UPF design information. This section describes about the following topics:

- [Abstract](#)
- [Quick Start](#)
- [Properties and Enum](#)
- [Classes](#)
- [L0 APIs](#)

Abstract

This model lets users get the power-management information from the Unified Power Format (UPF) language perspective. For NPI Upf Model, the IEEE Standard 1801-2015 UPF Language Reference Manual (LRM) is applied as the foundation of it. Most of the object diagrams are similar to the tables in LRM Section 10.6.

Refer to [L0 APIs](#) to see all the APIs that are used in the Upf Model. Refer to [Classes](#) to understand the properties and relationships of each object.

Quick Start

An example of how to run a pyNPI Upf APIs is as follows:

- [Quick Start](#)
 - [Environment and library setting](#)
 - [Example](#)
 - [Execution](#)
 - [Result](#)

Quick Start

This topic describes the following sections:

Environment and library setting

1. Update `LD_LIBRARY_PATH` to access NPI shared library put in Verdi release package. If there exists any error like “cannot open shared object file”, add the shared library path to `LD_LIBRARY_PATH` before running your Python-Based NPI program.

For example, you can set the following environment under 64-bit Linux machine by a C shell (csh) script:

```
setenv LD_LIBRARY_PATH
$VERDI_HOME/share/NPI/lib/linux64:$LD_LIBRARY_PATH
setenv LD_LIBRARY_PATH
$VERDI_HOME/platform/linux64/bin:$LD_LIBRARY_PATH
```

2. Add python library path by the following commands:

```
rel_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(rel_lib_path))
```

3. Import “npisys” to use the function of NPI initialization and exit.

Import “upf” to use the APIs of Upf Model.

Import libraries as follows:

```
from pynpi import npisys
from pynpi import upf
```

Example

example.v:

```
module flop(d,clk,reset,Q,restore,save);
    input d,clk,reset,restore,save;
    output reg Q;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            Q<=0;
        end
        else begin
            Q<=d;
        end
    end
end
endmodule

module top(d,clkFast,clkSlow,reset,QFast,QSlow,restore,save);
    input d,clkFast,clkSlow,reset,restore,save;
```

```

        output reg QFast,QSlow;
        flop fastFlop
        (.d(d),.clk(clkFast),.reset(reset),.Q(QFast),.restore(restore),.save(save));
        flop slowFlop
        (.d(d),.clk(clkSlow),.reset(reset),.Q(QSlow),.restore(restore),.save(save));
    endmodule

module testbench();
    reg clkFast,clkSlow,reset,d,restore,save;
    wire Qfast,QSlow;
    top
    a(.d(d),.clkFast(clkFast),.clkSlow(clkSlow),.reset(reset),.QFast(QFast),
    .QSlow(QSlow),.restore(restore),.save(save));
endmodule

```

example.upf:

```

set_design_top top
create_power_domain PD_TOP

set_retention RET1 -domain PD_TOP -save_signal {save high}
    -restore_signal {restore high}
set_retention RET2 -domain PD_TOP -save_signal {save high}
    -restore_signal {restore high} -elements {fastFlop}
set_retention RET3 -domain PD_TOP -save_signal {save high}
    -restore_signal {restore high} -elements {slowFlop}

```

example.py:

```

#!/usr/bin/env python3

import sys
import os
rel_lib_path = os.environ["VERDI_HOME"] + "/share/NPI/python"
sys.path.append(os.path.abspath(rel_lib_path))
from pynpi import npisys
from pynpi import upf

# Start to develop your code -----
# Function to dump handle information
def dump_hdl_info(hdl):
    # dump string properties
    print(" Name    = "+hdl.name())
    print(" Type    = "+hdl.type())
    print(" File     = "+hdl.file())
    # dump int properties
    print(" Line     = "+str(hdl.line()))

# Initialize NPI
if not npisys.init(sys.argv):

```

```

        print("Error: Fail to initialize NPI")
        assert 0

# Load design
if not npisys.load_design(sys.argv):
    print("Error: Fail to load design")
    assert 0

# Get top scope
print("==== Get top scope =====")
top_scope = upf.get_top_scope()
print(" Name   = "+top_scope.name())
print(" Type   = "+top_scope.type())

# Get all power domains
print("\n==== Get all power domains =====")
pd_list = upf.get_power_domains()
for pd in pd_list:
    print("Power domain: "+pd.full_name())
    dump_hdl_info(pd)

# Get specified handle by specific name
print("\n==== Get specified handle by full name =====")
PD_TOP = upf.handle_by_name("testbench/a/PD_TOP", None)
print("Get PD_TOP by \"handle_by_name\": "+PD_TOP.full_name())
dump_hdl_info(PD_TOP)

primary = upf.handle_by_name("primary", PD_TOP)
print("Get primary by \"handle_by_name\": "+primary.full_name())
dump_hdl_info(primary)

# 1-1 relationship
print("\n==== 1-1 relationship =====")
parent = primary.parent()
print("Get parent handle of primary: "+parent.full_name())
dump_hdl_info(parent)

# 1-m relationship
print("\n==== 1-m relationship =====")
retention_strategies = parent.retention_strategies()
for retention in retention_strategies:
    print("Get retention strategy handle of PD_TOP: "+retention.full_name())
    dump_hdl_info(retention)

# general properties
print("\n==== general properties =====")
save_signal = upf.handle_by_name("testbench/a/PD_TOP.RET1.save_signal",
    None)
print("Get save_signal by \"handle_by_name\": "+save_signal.full_name())
# string properties
print(" Name       = "+save_signal.name())
print(" File       = "+save_signal.file())

```

```
# int properties
print(" Line      = "+str(save_signal.line()))
# enum / string properties
print(" Type      =
      "+save_signal.type()+" (" +str(save_signal.type(True))+") ")
# int / string properties
print(" Ref Kind  =
      "+save_signal.ref_kind()+" (" +str(save_signal.ref_kind(True))+") ")
# End of your code -----

# End NPI
npisys.end()
```

run_demo:

```
#!/bin/csh -f

# You can also setup your $VERDI_HOME and $VCS_HOME here
# setup NPI shared library path
setenv LD_LIBRARY_PATH $VERDI_HOME/share/NPI/lib/linux64:$LD_LIBRARY_PATH
setenv LD_LIBRARY_PATH $VERDI_HOME/platform/linux64/bin:$LD_LIBRARY_PATH
# compile the design
$VCS_HOME/bin/vcs -full64 -sverilog example.v -upf example.upf
    -power=upfim -kdb
# run the python program
./example.py -dbdir simv.daidir
```

Execution

```
./run_demo
```

Result

```
===== Get top scope =====
Name      = testbench/a
Type      = npiVarPowerScope

===== Get all power domains =====
Power domain: testbench/a/PD_TOP
Name      = PD_TOP
Type      = npiVarPowerDomain
File      = ./example.upf
Line      = 2

===== Get specified handle by full name =====
Get PD_TOP by "handle_by_name": testbench/a/PD_TOP
Name      = PD_TOP
Type      = npiVarPowerDomain
File      = ./example.upf
Line      = 2
Get primary by "handle_by_name": testbench/a/PD_TOP.primary
Name      = primary
Type      = npiVarSupplySet
```

```
File      = ./example.upf
Line      = 2

===== 1-1 relationship =====
Get parent handle of primary: testbench/a/PD_TOP
Name      = PD_TOP
Type      = npfUpfPowerDomain
File      = ./example.upf
Line      = 2

===== 1-m relationship =====
Get retention strategy handle of PD_TOP: testbench/a/PD_TOP.RET1
Get retention strategy handle of PD_TOP: testbench/a/PD_TOP.RET2
Get retention strategy handle of PD_TOP: testbench/a/PD_TOP.RET3
Name      = RET3
Type      = npfUpfRetentionStrategy
File      = ./example.upf
Line      = 6

===== general properties =====
Get save_signal by "handle_by_name": testbench/a/PD_TOP.RET1.save_signal
Name      = save_signal
File      = ./example.upf
Line      = 4
Type      = npfUpfNamedRef (ClassType.NamedRef)
Ref Kind  = npfUpfRefSaveSignal (7)
```

Properties and Enum

- All objects will have below common properties:

1. **type**: It shows the type of the Upf handle.

A `tagHdl` object's type is `ClassType.tag`, for example, a `PowerDomainHdl` object's type is `ClassType.PowerDomain`.

- A property of handle might return an integer or a string. If the property supports both integer and string, the option `get_enum` is used to control the output format.

For example, property `port_dir` defined by the following syntax:

```
def port_dir(self, get_enum=False)
```

An exception is the property `type`, it returns an `IntEnum` of `ClassType` when `get_enum` set True.

All the property strings and integer are listed in: [Property strings and integers](#).

Classes

All classes with properties and relationships between objects are listed in the following:

- [Enums](#)
 - [ClassType](#)
 - [Property strings and integers](#)
- [Class Sets](#)
 - [BaseNamedSet](#)
 - [NetworkClassSet](#)
- [PowerDomainHdl](#)
- [RetentionStrategyHdl](#)
- [IsolationStrategyHdl](#)
- [RepeaterStrategyHdl](#)
- [SupplySetHdl](#)
- [NamedRefHdl](#)
- [SupplyNetHdl](#)
- [SupplyPortHdl](#)
- [LogicNetHdl](#)
- [LogicPortHdl](#)
- [PowerSwitchHdl](#)
- [AckPortHdl](#)
- [PowerStateHdl](#)
- [PowerSwitchStateHdl](#)
- [SupplyPortStateHdl](#)
- [PstStateHdl](#)
- [PowerStateTableHdl](#)
- [ExtentHdl](#)
- [SignalSenseHdl](#)

- [ExpressionHdl](#)
- [PowerScopeHdl](#)

Enums

Enums contains following sections:

ClassType

class upf.ClassType

An enumeration.

```
AckPort = 1
Expression = 5
Extent = 6
IsolationStrategy = 13
LogicNet = 15
LogicPort = 16
NamedRef = 17
PowerDomain = 18
PowerScope = 19
PowerState = 20
PowerStateTable = 21
PowerSwitchState = 23
PowerSwitch = 24
PstState = 25
RepeaterStrategy = 26
RetentionStrategy = 27
SignalSense = 28
SupplyNet = 29
SupplyPort = 30
SupplyPortState = 31
SupplySet = 32
```

Property strings and integers

- For [IsolationStrategyHdl.applies_to\(get_enum=False\)](#),
[RepeaterStrategyHdl.applies_to\(get_enum=False\)](#):

string	integer	comment
npfUpfFilterUndef	0	Undefined, when no information is available
npfUpfFilterInputs	1	-applies_to inputs
npfUpfFilterOutputs	2	-applies_to outputs
npfUpfFilterBoth	3	-applies_to both

- For [IsolationStrategyHdl.clamp_value\(get_enum=False\)](#):

string	integer	comment
npiUpfClampUndef	0	Undefined, when no information is available
npiUpfClampZero	1	-*_clamp 0
npiUpfClampOne	2	-*_clamp 1
npiUpfClampZee	3	-*_clamp Z
npiUpfClampLatch	4	-*_clamp latch

- For [IsolationStrategyHdl.location\(get_enum=False\)](#),
[RepeaterStrategyHdl.location\(get_enum=False\)](#):

string	integer	comment
npiUpfSelf	0	-location self
npiUpfOther	1	-location other
npiUpfParent	2	-location parent
npiUpfAutomatic	3	-location automatic
npiUpfFanout	4	-location fanout

- For [AckPortHdl.port_dir\(get_enum=False\)](#), [SupplyPortHdl.port_dir\(get_enum=False\)](#):

string	integer	comment
npiUpfDirUndef	0	Undefined, when no information is available
npiUpfDirIn	1	-direction in
npiUpfDirOut	2	-direction out
npiUpfDirInOut	3	-direction inout

- For [NamedRefHdl.ref_kind\(get_enum=False\)](#):

string	integer	comment
npiUpfRefPower	0	power function of supply set
npiUpfRefGround	1	ground function of supply set
npiUpfRefPwell	2	pwell function of supply set
npiUpfRefNwell	3	nwell function of supply set
npiUpfRefDeepwell	4	deeppwell function of supply set
npiUpfRefDeepnwell	5	deepnwell function of supply set
npiUpfRefIsoSignal	6	reference to isolation control signal in set_isolation
npiUpfRefSaveSignal	7	reference to save_signal in set_retention
npiUpfRefRestoreSignal	8	reference to restore_signal in set_retention
npiUpfRefGenericClock	9	reference to UPF_GENERIC_CLOCK in set_retention
npiUpfRefGenericData	10	reference to UPF_GENERIC_DATA in set_retention
npiUpfRefGenericAsyncLoad	11	reference to UPF_GENERIC_ASYNC_LOAD in set_retention
npiUpfRefGenericOutput	12	reference to UPF_GENERIC_OUTPUT in set_retention
npiUpfRefUserDefined	13	some user defined ref handle

- For [SignalSenseHdl.signal_sensitivity\(get_enum=False\)](#):

string	integer	comment
npiUpfSenseHigh	0	high
npiUpfSenseLow	1	low
npiUpfSensePosedge	2	posedge
npiUpfSenseNegedge	3	negedge

- For [PowerStateHdl.simstate\(get_enum=False\)](#):

string	integer	comment
npfUpfNormal	0	-simstate NORMAL
npfUpfCorrupt	1	-simstate CORRUPT or -simstate NOT_NORMAL
npfUpfCorruptOnActivity	2	-simstate CORRUPT_ON_ACTIVITY or -simstate NOT_NORMAL
npfUpfCorruptOnChange	3	-simstate CORRUPT_ON_CHANGE or -simstate NOT_NORMAL
npfUpfCorruptStateOnActivity	4	-simstate CORRUPT_STATE_ON_ACTIVITY or -simstate NOT_NORMAL
npfUpfCorruptStateOnChange	5	-simstate CORRUPT_STATE_ON_CHANGE or -simstate NOT_NORMAL

- For [SupplyPortStateHdl.supply_state\(get_enum=False\)](#),
[PowerSwitchStateHdl.switch_output_state\(get_enum=False\)](#):

string	integer	comment
npfUpfOff	0	-off_state or reference to OFF for supply net/port state
npfUpfUndetermined	1	-error_state or reference to UNDETERMINED for supply net/port state
npfUpfPartialOn	2	-on_partial_state or reference to PARTIAL_ON for supply net/port state
npfUpfFullOn	3	-on_state or reference to FULL_ON for supply net/port state

- For [SupplyPortStateHdl.volt_kind\(get_enum=False\)](#):

string	integer	comment
npfUpfState	0	when "off" is specified in add_port_state command
npfUpfNom	1	when only nominal value is specified in add_port_state command

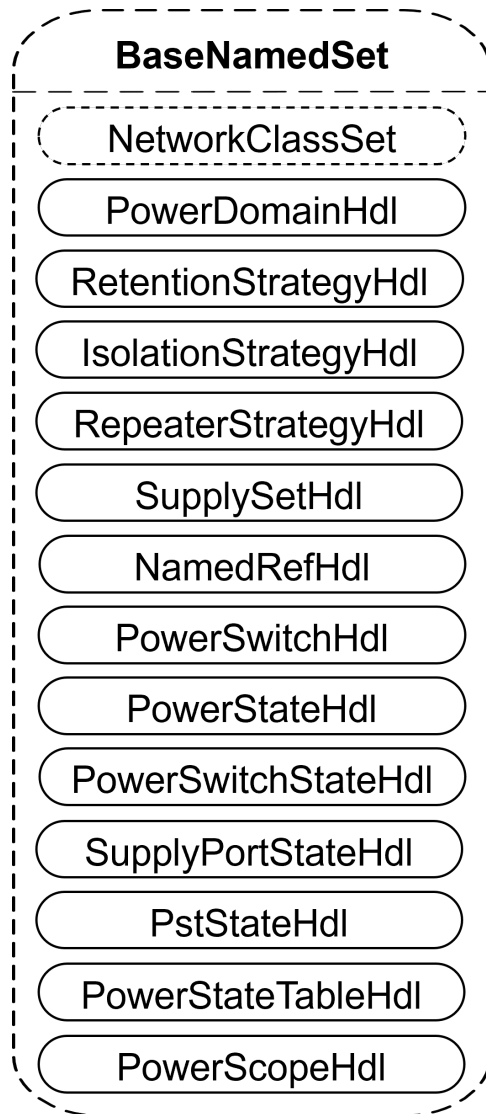
string	integer	comment
npiUpfDoublet	2	when a doublet is specified in add_port_state command
npiUpfTriplet	3	when a triplet is specified in add_port_state command

Class Sets

Class Sets contains following sections:

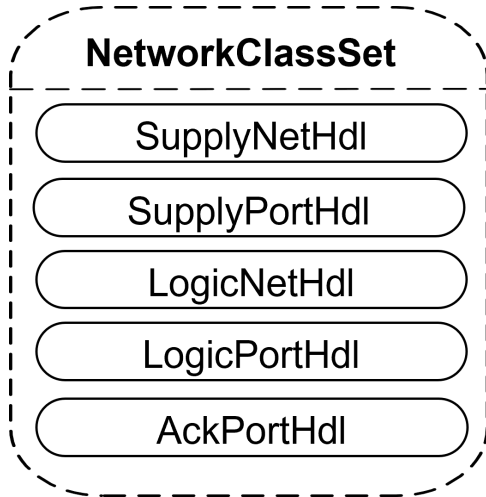
BaseNamedSet

Object diagram:



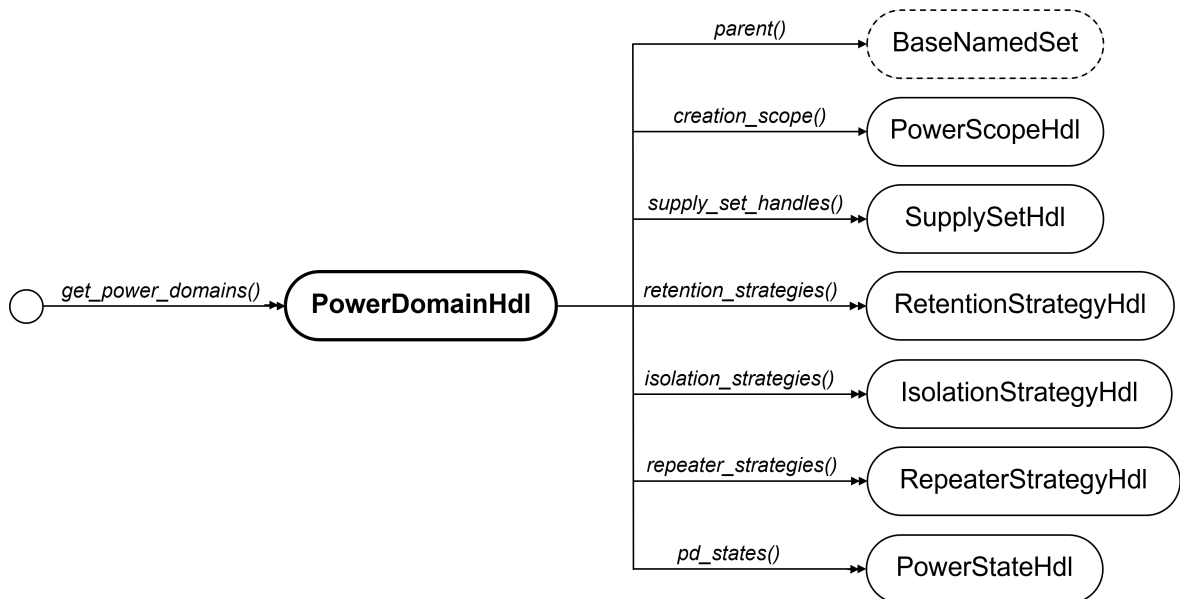
NetworkClassSet

Object diagram:



PowerDomainHdl

Object diagram:



```
class upf.PowerDomainHdl(cps_obj)
```

A `PowerDomainHdl` object is created when `create_power_domain` command is executed.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the power domain is created.

1-m method:

<code>supply_set_handles()</code>	Get supply set handles defined on the power domain in list.
<code>retention_strategies()</code>	Get retention strategy handles defined for the power domain in list.
<code>isolation_strategies()</code>	Get isolation strategy handles defined for the power domain in list.
<code>repeater_strategies()</code>	Get repeater strategy handles defined for the power domain in list.
<code>pd_states()</code>	Get power state handles defined on the power domain in list.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the power domain exists.
<code>line()</code>	The line number in the file where the power domain exists.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the power domain is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

supply_set_handles()

Get supply set handles defined on the power domain in list.

Parameters: None. –

Returns: The list of supply set handles.

Return type: list

Examples

```
>>> hdl = ref.supply_set_handles()
```

retention_strategies()

Get retention strategy handles defined for the power domain in list.

Parameters: None. –

Returns: The list of retention strategy handles.

Return type: list

Examples

```
>>> hdl_list = ref.retention_strategies()
```

isolation_strategies()

Get isolation strategy handles defined for the power domain in list.

Parameters: None. –

Returns: The list of isolation strategy handles.

Return type: list

Examples

```
>>> hdl_list = ref.isolation_strategies()
```

repeater_strategies()

Get repeater strategy handles defined for the power domain in list.

Parameters: None. –

Returns: The list of repeater strategy handles.

Return type: list

Examples

```
>>> hdl_list = ref.repeater_strategies()
```

pd_states()

Get power state handles defined on the power domain in list.

Parameters: None. –

Returns: The list of power state handles.

Return type: list

Examples

```
>>> hdl_list = ref.pd_states()
```

type(*get_enum=False*)

The type string/enum.

Parameters: *get_enum*(*bool*) – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the power domain exists.

Parameters: None. –

Returns: The file path of the power domain.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the power domain exists.

Parameters: None. –

Returns: The line number of the power domain.

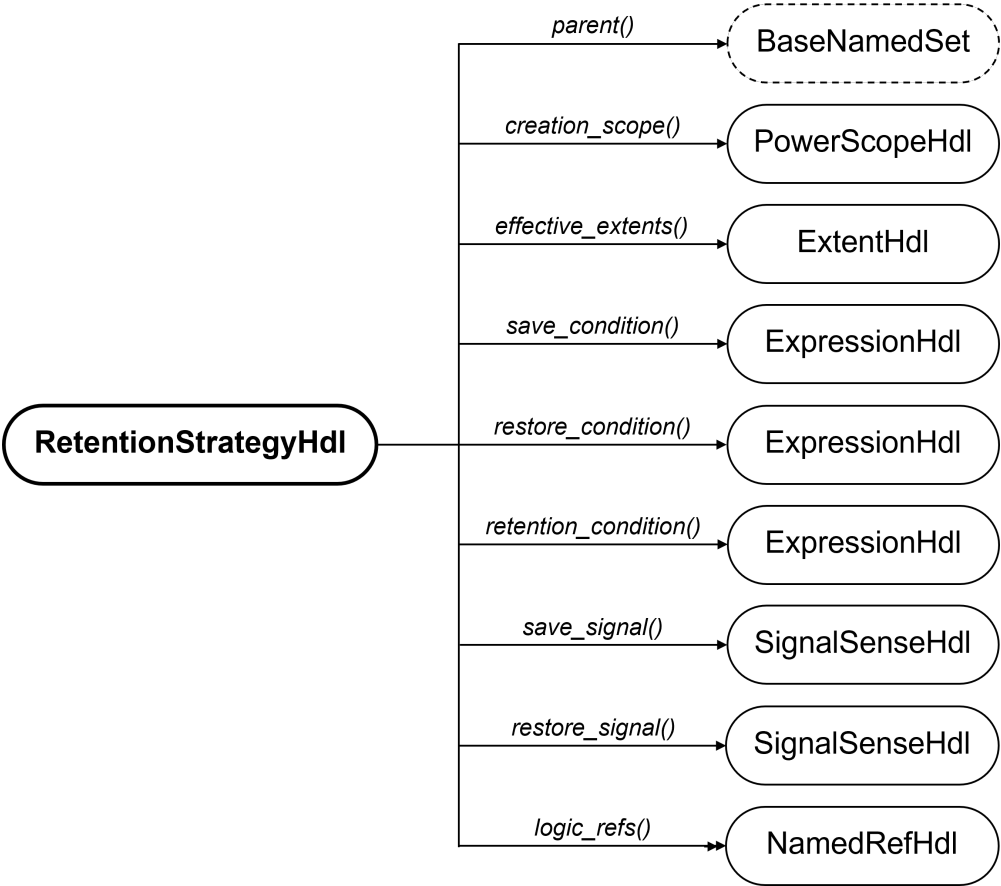
Return type: int

Examples

```
>>> hdl.line()
```

RetentionStrategyHdl

Object diagram:



```
class upf.RetentionStrategyHdl(cps_obj)
```

A `RetentionStrategyHdl` object is created when `set_retention` command is executed.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the retention strategy is created.
<code>effective_extents()</code>	Get extent handle representing the first element to which the retention strategy applies.
<code>save_condition()</code>	Get expression handle representing -save_condition information.

restore_condition()	Get expression handle representing -restore_condition information.
retention_condition()	Get expression handle representing -retention_condition information.
save_signal()	Get control sensitivity handle representing -save_signal information.
restore_signal()	Get control sensitivity handle representing -restore_signal information.

1-m methods:

logic_refs()	Get named object reference handles defined for the retention strategy in list.
------------------------------	--

General Properties:

type(get_enum=False)	The type string/enum.
name()	The short name string.
full_name()	The full hierarchical name string.
file()	File path where the retention strategy exists.
line()	The line number in the file where the retention strategy exists.
is_no_retention()	Flag for -no_retention.
is_use_retention_as_primary()	Flag for -use_retention_as_primary.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the retention strategy is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

effective_extents()

Get extent handle representing the first element to which the retention strategy applies.

Parameters: None. –

Returns: The extent handle.

Return type: [ExtentHdl](#)

Examples

```
>>> hdl = ref.effective_extents()
```

save_condition()

Get expression handle representing `-save_condition` information.

Parameters: None. –

Returns: The expression handle.

Return type: [ExpressionHdl](#)

Examples

```
>>> hdl = ref.save_condition()
```

restore_condition()

Get expression handle representing `-restore_condition` information.

Parameters: None. –

Returns: The expression handle.

Return type: [ExpressionHdl](#)

Examples

```
>>> hdl = ref.restore_condition()
```

retention_condition()

Get expression handle representing `-retention_condition` information.

Parameters: None. –

Returns: The expression handle.

Return type: [ExpressionHdl](#)

Examples

```
>>> hdl = ref.retention_condition()
```

save_signal()

Get control sensitivity handle representing `-save_signal` information.

Parameters: None. –

Returns: The control sensitivity handle.

Return type: [SignalSenseHdl](#)

Examples

```
>>> hdl = ref.save_signal()
```

restore_signal()

Get control sensitivity handle representing `-restore_signal` information.

Parameters: None. –

Returns: The control sensitivity handle.

Return type: [SignalSenseHdl](#)

Examples

```
>>> hdl = ref.restore_signal()
```

logic_refs()

Get named object reference handles defined for the retention strategy in list.

Parameters: None. –

Returns: The list of named object reference handles.

Return type: list

Examples

```
>>> hdl = ref.logic_refs()
```

type(*get_enum=False*)

The type string/enum.

Parameters: *get_enum(bool)* – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the retention strategy exists.

Parameters: None. –

Returns: The file path of the retention strategy.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the retention strategy exists.

Parameters: None. –

Returns: The line number of the retention strategy.

Return type: int

Examples

```
>>> hdl.line()
```

is_no_retention()

Flag for -no_retention.

Parameters: None. –

Returns: if -no_retention is used, return 1; otherwise, return 0.

Return type: int

Examples

```
>>> hdl.is_no_retention()
```

is_use_retention_as_primary()

Flag for -use_retention_as_primary.

Parameters: None. –

Returns: if -use_retention_as_primary is used, return 1; otherwise, return 0.

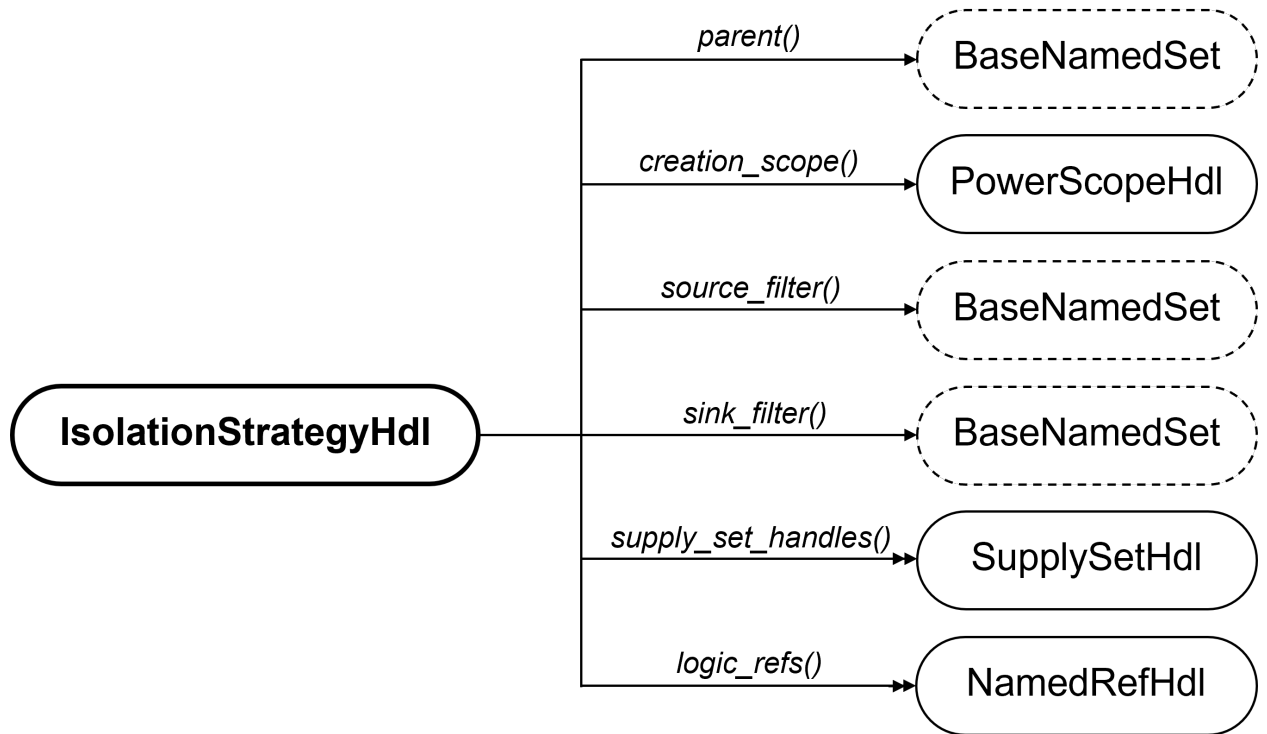
Return type: int

Examples

```
>>> hdl.is_use_retention_as_primary()
```


IsolationStrategyHdl

Object diagram:



```
class upf.IsolationStrategyHdl(cps_obj)
```

A **IsolationStrategyHdl** object is created when **set_isolation** command is executed.

1-1 method:

parent()	Get parent handle.
creation_scope()	Get power scope handle in which the isolation strategy is created.
source_filter()	Get handle representing -source filter information.
sink_filter()	Get handle representing -sink filter information.

1- m methods:

supply_set_handles()	Get supply set handles defined on the isolation strategy in list.
--------------------------------------	---

<code>logic_refs()</code>	Get named object reference handles defined for the isolation strategy in list.
---------------------------	--

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the isolation strategy exists.
<code>line()</code>	The line number in the file where the isolation strategy exists.
<code>location(get_enum=False)</code>	The string/int representing -location information.
<code>applies_to(get_enum=False)</code>	The string/int representing -applies_to information.
<code>is_use_equivalence()</code>	Flag for -use_equivalence.
<code>is_diff_supply_only()</code>	Flag for -diff_supply_only.
<code>is_no_isolation()</code>	Flag for -no_isolation.
<code>clamp_value(get_enum=False)</code>	The string/int of -clamp_value information.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: `BaseNamedSet`

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the isolation strategy is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

source_filter()

Get handle representing -source filter information.

Parameters: None. –

Returns: The handle for source filter.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl = ref.source_filter()
```

sink_filter()

Get handle representing -sink filter information.

Parameters: None. –

Returns: The handle for sink filter.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl = ref.sink_filter()
```

supply_set_handles()

Get supply set handles defined on the isolation strategy in list.

Parameters: None. –

Returns: The list of supply set handles.

Return type: list

Examples

```
>>> hdl = ref.supply_set_handles()
```

logic_refs()

Get named object reference handles defined for the isolation strategy in list.

Parameters: None. –

Returns: The list of named object reference handles.

Return type: list

Examples

```
>>> hdl = ref.logic_refs()
```

type(*get_enum=False*)

The type string/enum.

Parameters: *get_enum(bool)* – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the isolation strategy exists.

Parameters: None. –

Returns: The file path of the isolation strategy.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the isolation strategy exists.

Parameters: None. –

Returns: The line number of the isolation strategy.

Return type: int

Examples

```
>>> hdl.line()
```

location(*get_enum=False*)

The string/int representing -location information.

Parameters: *get_enum(bool)* – Determine if return in integer. Default: False.

Returns: The string/int of location.

Return type: str/int

Examples

```
>>> hdl.location()                # Get string
>>> hdl.location(True)           # Get integer
```

applies_to(*get_enum=False*)

The string/int representing -applies_to information.

Parameters: *get_enum(bool)* – Determine if return in integer. Default: False.

Returns: The string/int of applies to.

Return type: str/int

Examples

```
>>> hdl.applies_to()           # Get string
>>> hdl.applies_to(True)      # Get integer
```

is_use_equivalence()

Flag for `-use_equivalence`.

Parameters: None. –

Returns: if `-use_equivalence` is used, return 1; otherwise, return 0.

Return type: int

Examples

```
>>> hdl.is_use_equivalence()
```

is_diff_supply_only()

Flag for `-diff_supply_only`.

Parameters: None. –

Returns: if `-diff_supply_only` is TRUE, return 1; otherwise, return 0.

Return type: int

Examples

```
>>> hdl.is_diff_supply_only()
```

is_no_isolation()

Flag for `-no_isolation`.

Parameters: None. –

Returns: if `no_isolation` is used, return 1; otherwise, return 0.

Return type: int

Examples

```
>>> hdl.is_no_isolation()
```

clamp_value(*get_enum=False*)

The string/int of `-clamp_value` information.

Parameters: *get_enum*(*bool*) – Determine if return in integer. Default: False.

Returns: The string/int of clamp value.

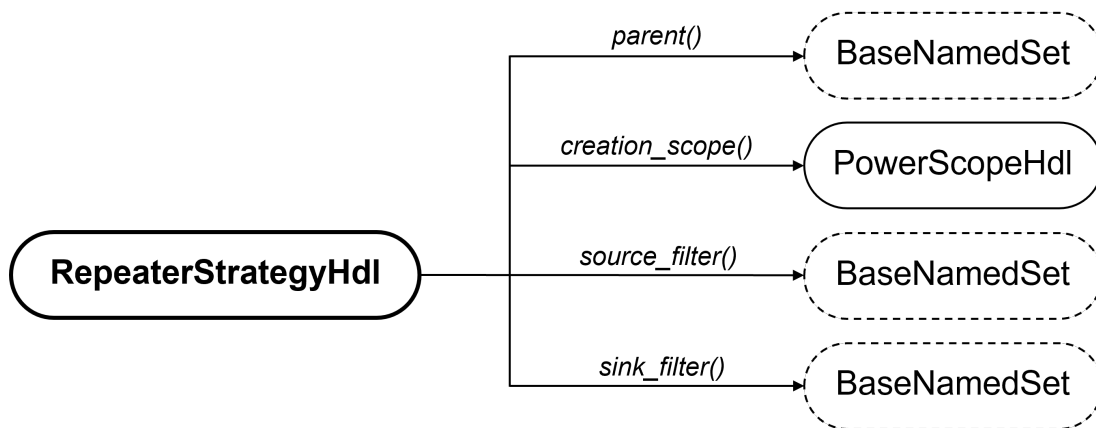
Return type: str/int

Examples

```
>>> hdl.clamp_value()           # Get string
>>> hdl.clamp_value(True)      # Get integer
```

RepeaterStrategyHdl

Object diagram:



```
class upf.RepeaterStrategyHdl(cps_obj)
```

A `RepeaterStrategyHdl` object is created when `set_repeater` command is executed.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the repeater strategy is created.
<code>source_filter()</code>	Get handle representing -source filter information.
<code>sink_filter()</code>	Get handle representing -sink filter information.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.

full_name()	The full hierarchical name string.
file()	File path where the repeater strategy exists.
line()	The line number in the file where the repeater strategy exists.
location(get_enum=False)	The string/int representing -location information.
applies_to(get_enum=False)	The string/int representing -applies_to information.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the repeater strategy is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

source_filter()

Get handle representing -source filter information.

Parameters: None. –

Returns: The handle for source filter.

Return type: [BaseNamedSet](#)

Examples


```
>>> hdl = ref.source_filter()
```

sink_filter()

Get handle representing -sink filter information.

Parameters: None. –

Returns: The handle for sink filter.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl = ref.sink_filter()
```

type(get_enum=False)

The type string/enum.

Parameters: *get_enum(bool)* – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)            # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the repeater strategy exists.

Parameters: None. –

Returns: The file path of the repeater strategy.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the repeater strategy exists.

Parameters: None. –

Returns: The line number of the repeater strategy.

Return type: int

Examples

```
>>> hdl.line()
```

location(*get_enum=False*)

The string/int representing -location information.

Parameters: *get_enum*(bool) – Determine if return in integer. Default: False.

Returns: The string/int of location.

Return type: str/int

Examples

```
>>> hdl.location()                # Get string
>>> hdl.location(True)           # Get integer
```

applies_to(*get_enum=False*)

The string/int representing -applies_to information.

Parameters: `get_enum(bool)` – Determine if return in integer. Default: False.

Returns: The string/int of applies to.

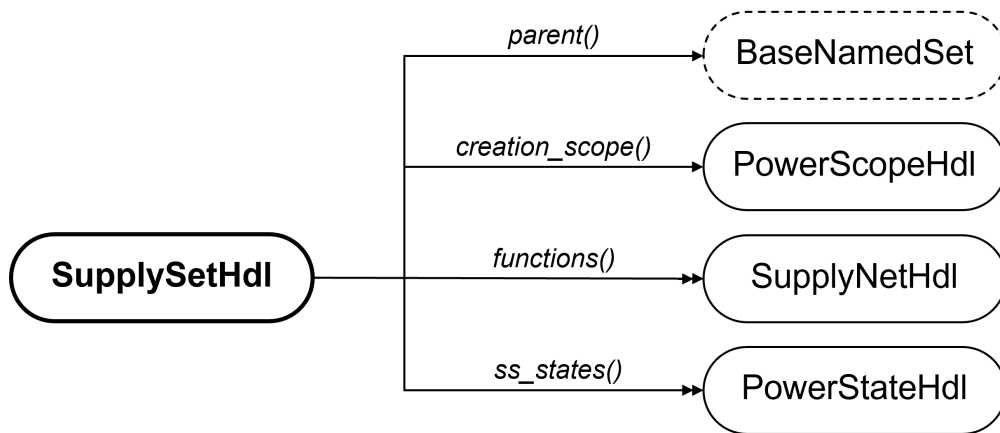
Return type: str/int

Examples

```
>>> hdl.applies_to()           # Get string
>>> hdl.applies_to(True)      # Get integer
```

SupplySetHdl

Object diagram:



```
class upf.SupplySetHdl(cps_obj)
```

A `SupplySetHdl` object is created when `create_supply_set` command is executed or for supply set handles presenting on other objects.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the supply set is created.

1-m methods:

<code>functions()</code>	Get supply net handles representing the functions of the supply set in list.
--------------------------	--

<code>ss_states()</code>	Get power state handles added by <code>add_power_state</code> command in list.
--------------------------	--

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the supply set exists.
<code>line()</code>	The line number in the file where the supply set exists.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the supply set is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

functions()

Get supply net handles representing the functions of the supply set in list.

Parameters: None. –

Returns: The list of supply net handles.

Return type: list

Examples

```
>>> hdl = ref.functions()
```

ss_states()

Get power state handles added by `add_power_state` command in list.

Parameters: None. –

Returns: The list of power state handles.

Return type: list

Examples

```
>>> hdl_list = ref.ss_states()
```

type(*get_enum=False*)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the supply set exists.

Parameters: None. –

Returns: The file path of the supply set.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the supply set exists.

Parameters: None. –

Returns: The line number of the supply set.

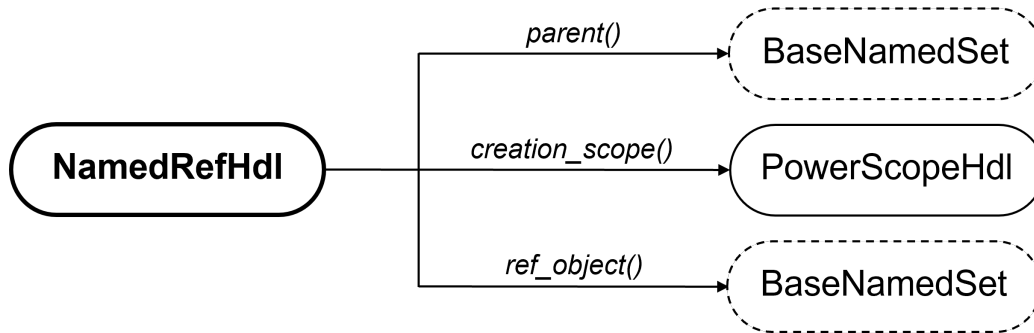
Return type: int

Examples

```
>>> hdl.line()
```

NamedRefHdl

Object diagram:



```
class upf.NamedRefHdl(cps_obj)
```

A `NamedRefHdl` object is created for logic references of strategies.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the named object reference is created.
<code>ref_object()</code>	Get handle representing reference to original object.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the named object reference exists.
<code>line()</code>	The line number in the file where the named object reference exists.
<code>ref_kind(get_enum=False)</code>	The string/int representing kind of reference.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the named object reference is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

ref_object()

Get handle representing reference to original object.

Parameters: None. –

Returns: The handle of original object.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl = ref.ref_object()
```

type(get_enum=False)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```


name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the named object reference exists.

Parameters: None. –

Returns: The file path of the named object reference.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the named object reference exists.

Parameters: None. –

Returns: The line number of the named object reference.

Return type: int

Examples

```
>>> hdl.line()
```

ref_kind(*get_enum=False*)

The string/int representing kind of reference.

Parameters: *get_enum(bool)* – Determine if return in integer. Default: False.

Returns: The string/int of reference kind.

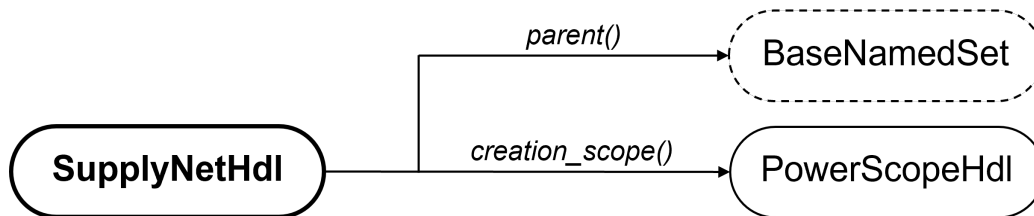
Return type: str/int

Examples

```
>>> hdl.ref_kind()                # Get string
>>> hdl.ref_kind(True)           # Get integer
```

SupplyNetHdl

Object diagram:



```
class upf.SupplyNetHdl(cps_obj)
```

A SupplyNetHdl object is created when create_supply_net command is executed.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the supply net is created.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the supply net exists.

<code>line()</code>	The line number in the file where the supply net exists.
---------------------	--

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the supply net is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

type(get_enum=False)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the supply net exists.

Parameters: None. –

Returns: The file path of the supply net.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the supply net exists.

Parameters: None. –

Returns: The line number of the supply net.

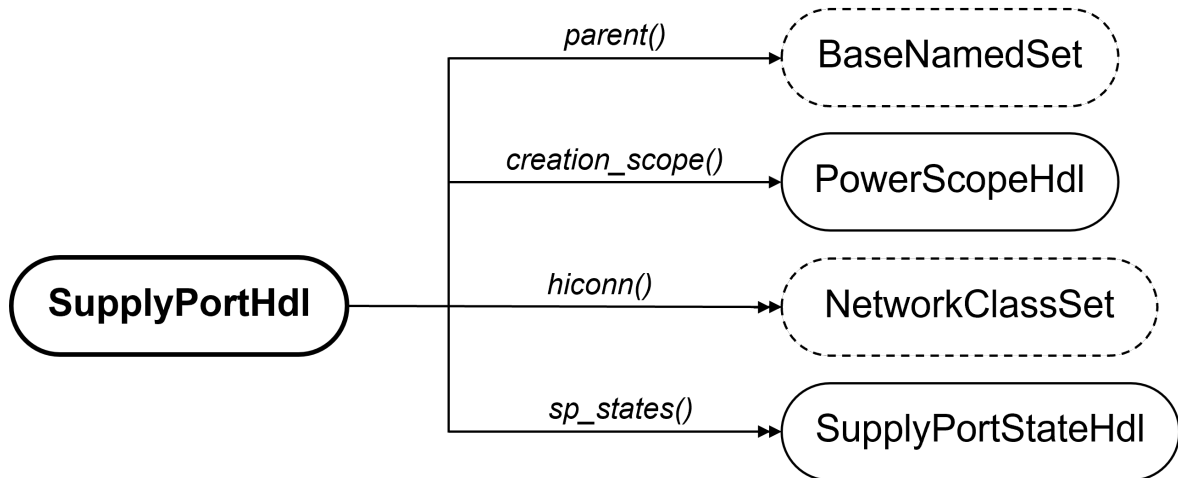
Return type: int

Examples

```
>>> hdl.line()
```

SupplyPortHdl

Object diagram:



```
class upf.SupplyPortHdl(cps_obj)
```

A **SupplyPortHdl** object is created when `create_supply_port` command is executed or `create_power_switch` creates the input/output supply ports.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the supply port is created.

1-m methods:

<code>hiconn()</code>	Get handles connected to the hiconn side of the supply port in list.
<code>sp_states()</code>	Get supply port state handles added by <code>add_port_state</code> command in list.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.

<code>file()</code>	File path where the supply port exists.
<code>line()</code>	The line number in the file where the supply port exists.
<code>port_dir(get_enum=False)</code>	The string/int representing direction of the supply port.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the supply port is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

hiconn()

Get handles connected to the hiconn side of the supply port in list.

Parameters: None. –

Returns: The list of handles in `NetworkClassSet`.

Return type: list

Examples

```
>>> hdl_list = ref.hiconn()
```

sp_states()

Get supply port state handles added by `add_port_state` command in list.

Parameters: None. –

Returns: The list of supply port state handles..

Return type: list

Examples

```
>>> hdl_list = ref.sp_states()
```

type(get_enum=False)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                                # Get string
>>> hdl.type(True)                            # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the supply port exists.

Parameters: None. –

Returns: The file path of the supply port.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the supply port exists.

Parameters: None. –

Returns: The line number of the supply port.

Return type: int

Examples

```
>>> hdl.line()
```

port_dir(*get_enum=False*)

The string/int representing direction of the supply port.

Parameters: *get_enum(bool)* – Determine if return in integer. Default: False.

Returns: The string/int of direction.

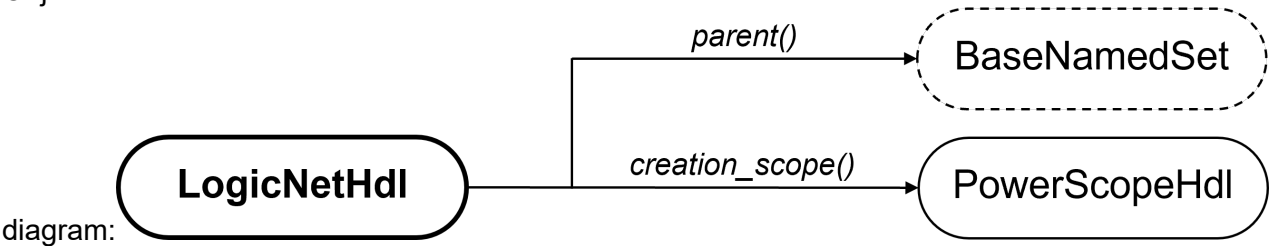
Return type: str/int

Examples

```
>>> hdl.port_dir()           # Get string
>>> hdl.port_dir(True)      # Get integer
```


LogicNetHdl

Object



`class upf.LogicNetHdl(cps_obj)`

A `LogicNetHdl` object is created when `create_logic_net` command is executed.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the logic net is created.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the logic net exists.
<code>line()</code>	The line number in the file where the logic net exists.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: `BaseNamedSet`

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the logic net is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

type(*get_enum=False*)

The type string/enum.

Parameters: *get_enum*(*bool*) – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                                # Get string
>>> hdl.type(True)                            # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the logic net exists.

Parameters: None. –

Returns: The file path of the logic net.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the logic net exists.

Parameters: None. –

Returns: The line number of the logic net.

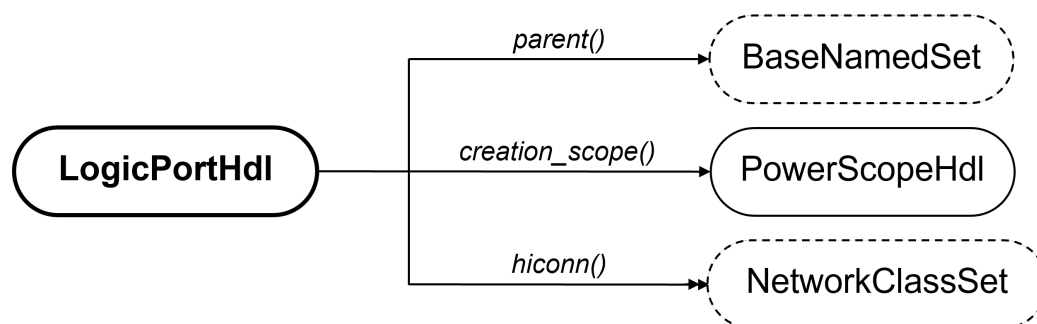
Return type: int

Examples

```
>>> hdl.line()
```

LogicPortHdl

Object diagram:



```
class upf.LogicPortHdl(cps_obj)
```

A **LogicPortHdl** object is created when `create_logic_port` command is executed or `create_power_switch` creates a control port.

1-1 method:

parent()	Get parent handle.
creation_scope()	Get power scope handle in which the logic port is created.

1-m methods:

hiconn()	Get handles connected to the hiconn side of the logic port in list.
--------------------------	---

General Properties:

type(get_enum=False)	The type string/enum.
name()	The short name string.
full_name()	The full hierarchical name string.
file()	File path where the logic port exists.
line()	The line number in the file where the logic port exists.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the logic port is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

hiconn()

Get handles connected to the hiconn side of the logic port in list.

Parameters: None. –

Returns: The list of handles in `NetworkClassSet`.

Return type: list

Examples

```
>>> hdl_list = ref.hiconn()
```

type(*get_enum=False*)

The type string/enum.

Parameters: *get_enum(bool)* – Determine if return in enum `ClassType`. Default: False.

Returns: The string/enum of handle type.

Return type: `ClassType`

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the logic port exists.

Parameters: None. –

Returns: The file path of the logic port.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the logic port exists.

Parameters: None. –

Returns: The line number of the logic port.

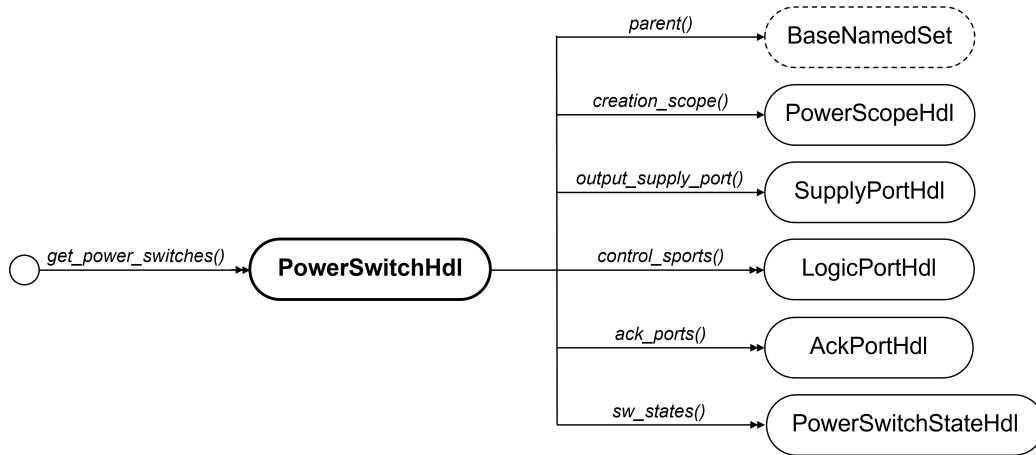
Return type: int

Examples

```
>>> hdl.line()
```

PowerSwitchHdl

Object diagram:



```
class upf.PowerSwitchHdl(cps_obj)
```

A **PowerSwitchHdl** object is created when `create_power_switch` command is executed.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the power switch is created.
<code>output_supply_port()</code>	Get supply port handle created by <code>-output_supply_port</code> option of <code>create_power_switch</code> command.

1-m methods:

<code>control_ports()</code>	Get logic port handles created by <code>-control_port</code> option of <code>create_power_switch</code> command in list.
<code>ack_ports()</code>	Get ack port handles created by <code>-ack_port</code> option of <code>create_power_switch</code> command in list.
<code>sw_states()</code>	Get list of power switch state handles created by <code>-on_state</code> , <code>-on_partial_state</code> , <code>-off_state</code> , and <code>-error_state</code> options of <code>create_power_switch</code> command.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the power switch exists.
<code>line()</code>	The line number in the file where the power switch exists.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the power switch is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

output_supply_port()

Get supply port handle created by `-output_supply_port` option of `create_power_switch` command.

Parameters: None. –

Returns: The supply port handle.

Return type: [SupplyPortHdl](#)

Examples

```
>>> hdl = ref.output_supply_port()
```

control_ports()

Get logic port handles created by `-control_port` option of `create_power_switch` command in list.

Parameters: None. –

Returns: The list of logic port handles.

Return type: list

Examples

```
>>> hdl_list = ref.control_ports()
```

ack_ports()

Get ack port handles created by `-ack_port` option of `create_power_switch` command in list.

Parameters: None. –

Returns: The list of ack port handles.

Return type: list

Examples

```
>>> hdl_list = ref.ack_ports()
```

sw_states()

Get list of power switch state handles created by `-on_state`, `-on_partial_state`, `-off_state`, and `-error_state` options of `create_power_switch` command.

Parameters: None. –

Returns: The list of power switch state handles.

Return type: list

Examples

```
>>> hdl_list = ref.sw_states()
```

type(*get_enum=False*)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum `ClassType`. Default: False.

Returns: The string/enum of handle type.

Return type: `ClassType`

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the power switch exists.

Parameters: None. –

Returns: The file path of the power switch.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the power switch exists.

Parameters: None. –

Returns: The line number of the power switch.

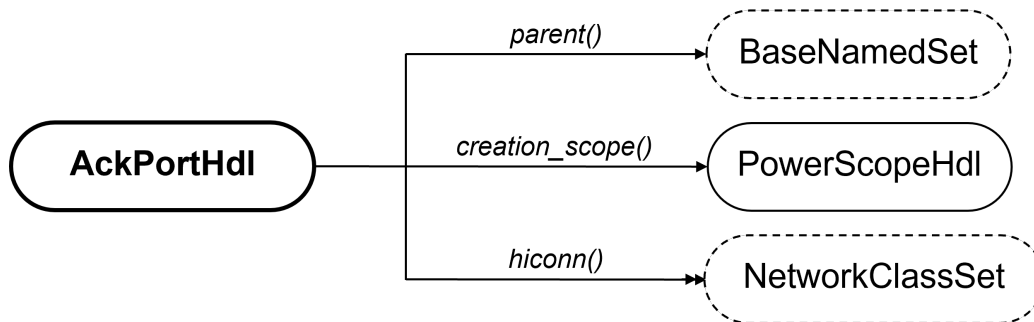
Return type: int

Examples

```
>>> hdl.line()
```

AckPortHdl

Object diagram:



```
class upf.AckPortHdl(cps_obj)
```

A **AckPortHdl** object is created when the `create_power_switch` command is executed with the `-ack_port` option specified.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the ack port is created.

1-m methods:

<code>hiconn()</code>	Get handles connected to the hiconn side of the ack port in list.
-----------------------	---

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the ack port exists.
<code>line()</code>	The line number in the file where the ack port exists.
<code>port_dir(get_enum=False)</code>	The string/int of direction of the ack port.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the ack port is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

hiconn()

Get handles connected to the hiconn side of the ack port in list.

Parameters: None. –

Returns: The list of handles in `NetworkClassSet`.

Return type: list

Examples

```
>>> hdl_list = ref.hiconn()
```

type(get_enum=False)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the ack port exists.

Parameters: None. –

Returns: The file path of the ack port.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the ack port exists.

Parameters: None. –

Returns: The line number of the ack port.

Return type: int

Examples

```
>>> hdl.line()
```

port_dir(*get_enum=False*)

The string/int of direction of the ack port.

Parameters: *get_enum(bool)* – Determine if return in integer. Default: False.

Returns: The string/int of direction.

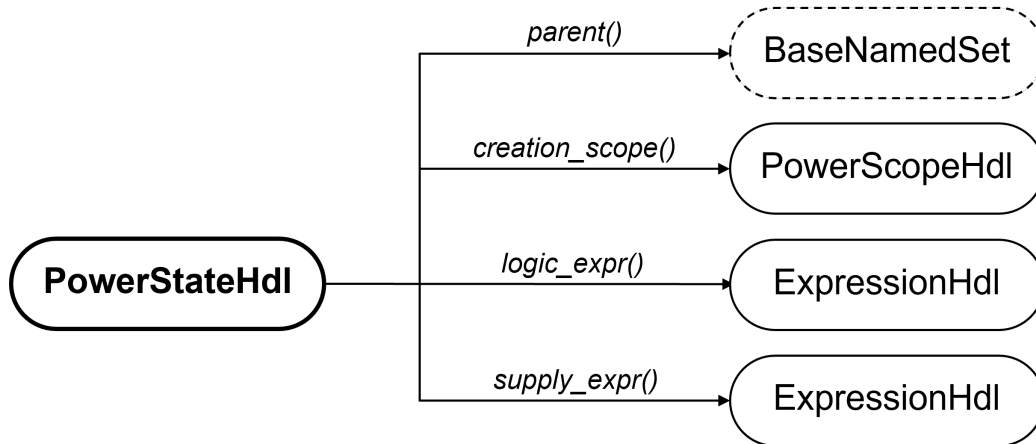
Return type: str/int

Examples

```
>>> hdl.port_dir()                # Get string
>>> hdl.port_dir(True)           # Get integer
```

PowerStateHdl

Object diagram:



```
class upf.PowerStateHdl(cps_obj)
```

A **PowerStateHdl** object is created when `add_power_state` command is executed.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the power state is created.
<code>logic_expr()</code>	Get expression handle specified by <code>-logic_expr</code> option of <code>add_power_state</code> command.
<code>supply_expr()</code>	Get expression handle specified by <code>-supply_expr</code> option of <code>add_power_state</code> command.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the power state exists.
<code>line()</code>	The line number in the file where the power state exists.

<code>is_illegal()</code>	Flag for -illegal.
<code>simstate(get_enum=False)</code>	The string/int representing simstate of the power state.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the power state is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

logic_expr()

Get expression handle specified by `-logic_expr` option of `add_power_state` command.

Parameters: None. –

Returns: The expression handle.

Return type: [ExpressionHdl](#)

Examples

```
>>> hdl = ref.logic_expr()
```

supply_expr()

Get expression handle specified by `-supply_expr` option of `add_power_state` command.

Parameters: None. –

Returns: The expression handle.

Return type: [ExpressionHdl](#)

Examples

```
>>> hdl = ref.supply_expr()
```

type(*get_enum=False*)

The type string/enum.

Parameters: *get_enum(bool)* – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the power state exists.

Parameters: None. –

Returns: The file path of the power state.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the power state exists.

Parameters: None. –

Returns: The line number of the power state.

Return type: int

Examples

```
>>> hdl.line()
```

is_illegal()

Flag for `-illegal`.

Parameters: None. –

Returns: if `-illegal` is used, return 1; otherwise, return 0.

Return type: int

Examples

```
>>> hdl.is_illegal()
```

simstate(*get_enum=False*)

The string/int representing simstate of the power state.

Parameters: `get_enum(bool)` – Determine if return in integer. Default: False.

Returns: The string/int of simstate.

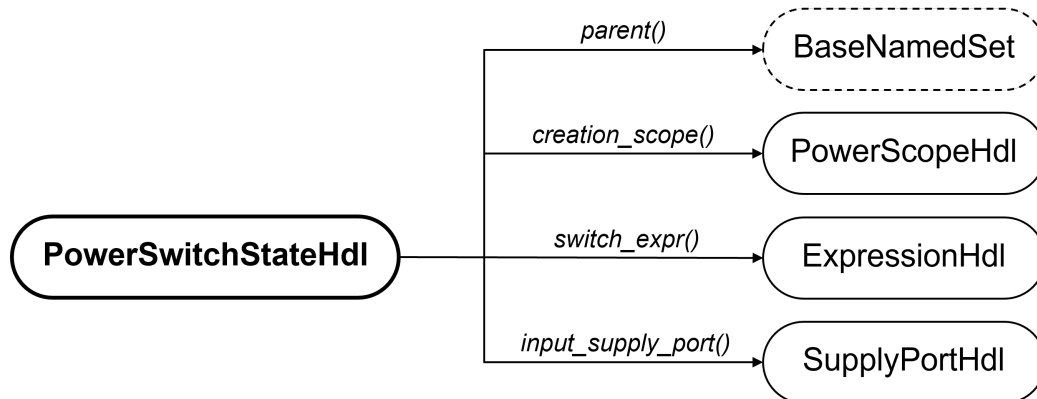
Return type: str/int

Examples

```
>>> hdl.simstate()           # Get string
>>> hdl.simstate(True)      # Get integer
```

PowerSwitchStateHdl

Object diagram:



```
class upf.PowerSwitchStateHdl(cps_obj)
```

A **PowerSwitchStateHdl** object is created when `create_power_switch` command is executed. The object is created to capture information about `-on_state`, `-on_partial_state`, `-off_state` and `-error_state` options.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the power switch state is created.
<code>switch_expr()</code>	Get expression handle specified in <code>-on_state</code> , <code>-on_partial_state</code> , <code>-off_state</code> and <code>-error_state</code> options of <code>create_power_switch</code> command
<code>input_supply_port()</code>	Get handle of input supply port connected when the state is on or <code>parital_on</code> .

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.

<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the power switch state exists.
<code>line()</code>	The line number in the file where the power switch state exists.
<code>switch_output_state(get_enum=False)</code>	The string/int representing state of the output of the power switch.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the power switch state is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

switch_expr()

Get expression handle specified in `-on_state`, `-on_partial_state`, `-off_state` and `-error_state` options of `create_power_switch` command

Parameters: None. –

Returns: The expression handle.

Return type: [ExpressionHdl](#)

Examples

```
>>> hdl = ref.switch_expr()
```

input_supply_port()

Get handle of input supply port connected when the state is on or `parital_on`.

Parameters: None. –

Returns: The supply port handle.

Return type: [SupplyPortHdl](#)

Examples

```
>>> hdl = ref.input_supply_port()
```

type(*get_enum=False*)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                                # Get string
>>> hdl.type(True)                            # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the power switch state exists.

Parameters: None. –

Returns: The file path of the power switch state.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the power switch state exists.

Parameters: None. –

Returns: The line number of the power switch state.

Return type: int

Examples

```
>>> hdl.line()
```

is_illegal()

Whether the state is created from `-error_state` of `create_power_switch` command.

Parameters: None. –

Returns: if true, return 1; otherwise, return 0.

Return type: int

Examples

```
>>> hdl.is_illegal()
```

switch_output_state(*get_enum=False*)

The string/int representing state of the output of the power switch.

Parameters: `get_enum(bool)` – Determine if return in integer. Default: False.

Returns: The string/int of state.

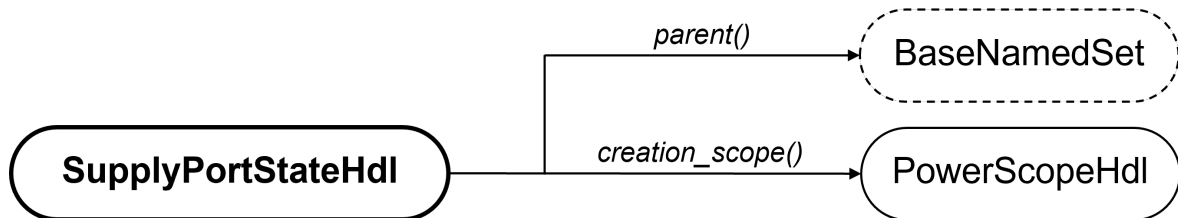
Return type: str/int

Examples

```
>>> hdl.switch_output_state()           # Get string
>>> hdl.switch_output_state(True)      # Get integer
```

SupplyPortStateHdl

Object diagram:



```
class upf.SupplyPortStateHdl(cps_obj)
```

A `SupplyPortStateHdl` object is created when `add_port_state` command is executed.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the supply port state is created.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the supply port state exists.
<code>line()</code>	The line number in the file where the supply port state exists.
<code>supply_state(get_enum=False)</code>	Get supply state string/int.
<code>volt_min()</code>	The minimum voltage value.
<code>volt_nom()</code>	The nominal voltage value.

<code>volt_max()</code>	The maximum voltage value.
<code>volt_kind(get_enum=False)</code>	The string/int representing whether nominal value, doublet or triplet is specified.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the supply port state is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

type(get_enum=False)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the supply port state exists.

Parameters: None. –

Returns: The file path of the supply port state.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the supply port state exists.

Parameters: None. –

Returns: The line number of the supply port state.

Return type: int

Examples

```
>>> hdl.line()
```

supply_state(*get_enum=False*)

Get supply state string/int.

Parameters: **get_enum**(*bool*) – Determine if return in integer. Default: False.

Returns: The string/int of state. If voltage value is specified, the supply state is `npIUpfFullOn`; if off is specified, the supply state is `npIUpfOff`.

Return type: str/int

Examples

```
>>> hdl.supply_state()           # Get string
>>> hdl.supply_state(True)      # Get integer
```

volt_min()

The minimum voltage value.

Parameters: None. –

Returns: The value `int(your value * (10^6))`.

Return type: int

Examples

```
>>> hdl.volt_min()
```

volt_nom()

The nominal voltage value.

Parameters: None. –

Returns: The value `int(your value * (10^6))`.

Return type: int

Examples

```
>>> hdl.volt_nom()
```

volt_max()

The maximum voltage value.

Parameters: None. –

Returns: The value `int(your value * (10^6))`.

Return type: int

Examples

```
>>> hdl.volt_max()
```

volt_kind(get_enum=False)

The string/int representing whether nominal value, doublet or triplet is specified.

Parameters: `get_enum(bool)` – Determine if return in integer. Default: False.

Returns: The string/int of voltage kind.

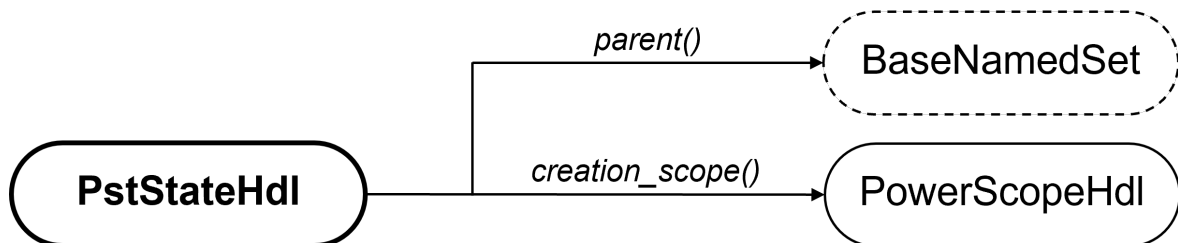
Return type: str/int

Examples

```
>>> hdl.volt_kind()                # Get string
>>> hdl.volt_kind(True)           # Get integer
```

PstStateHdl

Object diagram:



```
class upf.PstStateHdl(cps_obj)
```

A `PstStateHdl` object is created when `add_pst_state` command is executed.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the PST state is created.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.

<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the PST state exists.
<code>line()</code>	The line number in the file where the PST state exists.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the PST state is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

type(get_enum=False)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the PST state exists.

Parameters: None. –

Returns: The file path of the PST state.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the PST state exists.

Parameters: None. –

Returns: The line number of the PST state.

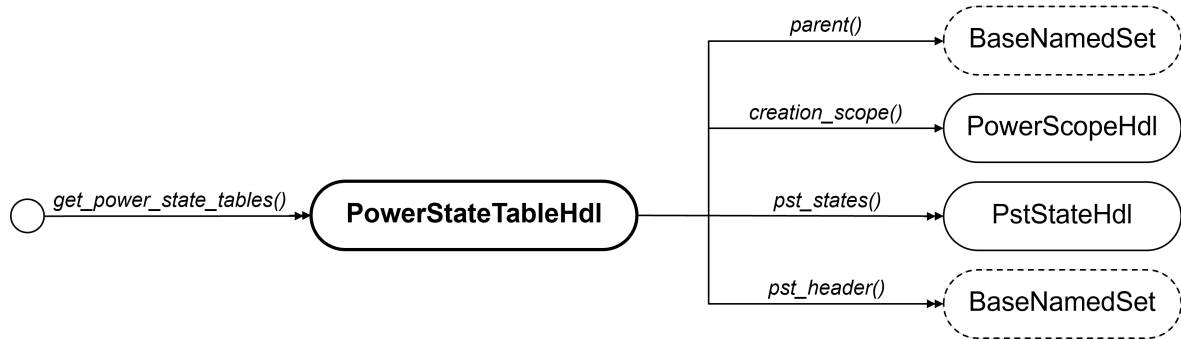
Return type: int

Examples

```
>>> hdl.line()
```

PowerStateTableHdl

Object diagram:



```
class upf.PowerStateTableHdl(cps_obj)
```

A `PowerStateTableHdl` object is created when `create_pst` command is executed.

1-1 method:

<code>parent()</code>	Get parent handle.
<code>creation_scope()</code>	Get power scope handle in which the power state table is created.

1-m methods:

<code>pst_states()</code>	Get PST state handles created by <code>add_pst_state</code> command in list.
<code>pst_header()</code>	Get handles forming column of the PST in list.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.
<code>file()</code>	File path where the power state table exists.

`line()`

The line number in the file where the power state table exists.

parent()

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

creation_scope()

Get power scope handle in which the power state table is created.

Parameters: None. –

Returns: The handle of power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> hdl = ref.creation_scope()
```

pst_states()

Get PST state handles created by `add_pst_state` command in list.

Parameters: None. –

Returns: The list of PST state handles.

Return type: list

Examples

```
>>> hdl_list = ref.pst_states()
```

pst_header()

Get handles forming column of the PST in list.

Parameters: None. –

Returns: The list of handles in [BaseNamedSet](#).

Return type: list

Examples

```
>>> hdl_list = ref.pst_header()
```

type(*get_enum=False*)

The type string/enum.

Parameters: *get_enum(bool)* – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                                # Get string
>>> hdl.type(True)                           # Get IntEnum
```

name()

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

file()

File path where the power state table exists.

Parameters: None. –

Returns: The file path of the power state table.

Return type: str

Examples

```
>>> hdl.file()
```

line()

The line number in the file where the power state table exists.

Parameters: None. –

Returns: The line number of the power state table.

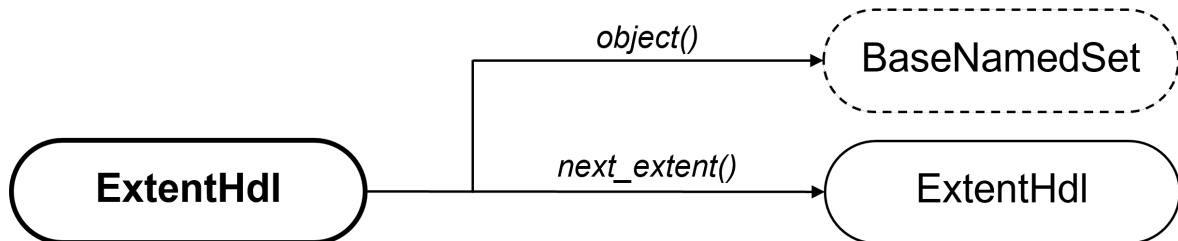
Return type: int

Examples

```
>>> hdl.line()
```

ExtentHdl

Object diagram:



```
class upf.ExtentHdl(cps_obj)
```

A `ExtentHdl` object is created to capture the information about the extent of power domains and strategies.

1-1 method:

<code>object()</code>	Get handle for which the extent is created.
<code>next_extent()</code>	Get extent handle representing the next element to which the command applies.

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
-----------------------------------	-----------------------

object()

Get handle for which the extent is created.

Parameters: None. –

Returns: The handle for which the extent is created.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl = ref.object()
```

next_extent()

Get extent handle representing the next element to which the command applies.

Parameters: None. –

Returns: The extent handle.

Return type: [ExtentHdl](#)

Examples

```
>>>hdl = ref.next_extent()
```

type(get_enum=False)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

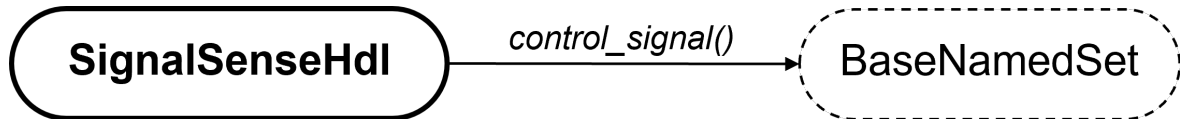
Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

SignalSenseHdl

Object diagram:



```
class upf.SignalSenseHdl(cps_obj)
```

A `SignalSenseHdl` object is created to represent the relationship between control signals and their sensitivity.

1-1 method:

<code>control_signal()</code>	Get handle representing control signal.
-------------------------------	---

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>signal_sensitivity(get_enum=False)</code>	The string/int representing sensitivity of control signal.

control_signal()

Get handle representing control signal.

Parameters: None. –

Returns: The handle of control signal.

Return type: `BaseNamedSet`

Examples

```
>>> hdl = ref.control_signal()
```

type(get_enum=False)

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum `ClassType`. Default: False.

Returns: The string/enum of handle type.

Return type: `ClassType`

Examples

```
>>> hdl.type() # Get string
>>> hdl.type(True) # Get IntEnum
```

signal_sensitivity(*get_enum=False*)

The string/int representing sensitivity of control signal.

Parameters: *get_enum(bool)* – Determine if return in integer. Default: False.

Returns: The string/int of sensitivity.

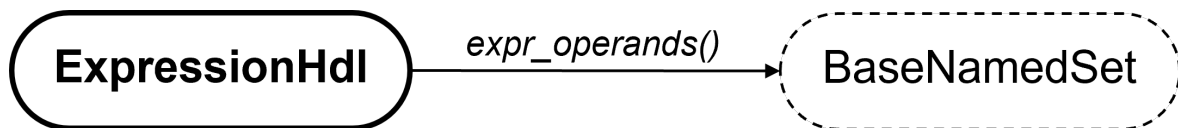
Return type: str/int

Examples

```
>>> hdl.signal_sensitivity() # Get string
>>> hdl.signal_sensitivity(True) # Get integer
```

ExpressionHdl

Object diagram:



```
class upf.ExpressionHdl(cps_obj)
```

A `ExpressionHdl` object is created to capture the boolean expression information.

1-1 method:

<code>expr_operands()</code>	Get handles which are operands used in the expression in list.
------------------------------	--

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>expr_string()</code>	The string representation of the expression.

expr_operands()

Get handles which are operands used in the expression in list.

Parameters: None. –

Returns: The list of handles in `BaseNamedSet`.

Return type: list

Examples

```
>>> hdl_list = ref.expr_operands()
```

`type(get_enum=False)`

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum `ClassType`. Default: False.

Returns: The string/enum of handle type.

Return type: `ClassType`

Examples

```
>>> hdl.type()                                # Get string
>>> hdl.type(True)                            # Get IntEnum
```

`expr_string()`

The string representation of the expression.

Parameters: None. –

Returns: The string of the expression.

Return type: str

Examples

```
>>> hdl.expr_string()
```

PowerScopeHdl

Object diagram:



```
class upf.PowerScopeHdl(cps_obj)
```

A `PowerScopeHdl` object is created for scope.

1-1 method:

<code>parent()</code>	Get parent handle.
-----------------------	--------------------

General Properties:

<code>type(get_enum=False)</code>	The type string/enum.
<code>name()</code>	The short name string.
<code>full_name()</code>	The full hierarchical name string.

`parent()`

Get parent handle.

Parameters: None. –

Returns: The parent handle.

Return type: [BaseNamedSet](#)

Examples

```
>>> hdl=ref.parent()
```

`type(get_enum=False)`

The type string/enum.

Parameters: `get_enum(bool)` – Determine if return in enum [ClassType](#). Default: False.

Returns: The string/enum of handle type.

Return type: [ClassType](#)

Examples

```
>>> hdl.type()                # Get string
>>> hdl.type(True)           # Get IntEnum
```

`name()`

The short name string.

Parameters: None. –

Returns: The string of handle name.

Return type: str

Examples

```
>>> hdl.name()
```

full_name()

The full hierarchical name string.

Parameters: None. –

Returns: The string of handle hierarchical name.

Return type: str

Examples

```
>>> hdl.full_name()
```

L0 APIs

For all document of APIs, it uses below case:

- [Demo Case for Documents](#)
 - [Design](#)
- L0 APIs available in the NPI Upf Model.

upf.handle_by_name(pathname, relativeTo)	Get a handle to an object with a specific name.
upf.get_top_scope()	Get a handle to the top power scope.
upf.get_power_domains()	Get a list of all power domains.
upf.get_power_switches()	Get a list of all power switches.
upf.get_power_state_tables()	Get a list of all power state tables.

Demo Case for Documents

For all document of APIs, it uses below design as example:

Design

- demo.v:

```
module buffer (out,in);
```

```

    input [7:0] in;
    output [7:0] out;

    reg [7:0] out;

    always @(in)
        out <= in;

endmodule

module testbench();

    reg [7:0] in;
    wire [7:0] out1;
    wire [7:0] out2;

    reg Vdd_wire, V1_wire, V2_wire, Vss_wire;
    reg psw_ctrl;
    reg psw_ctrl2;
    reg psw_ack;
    reg psw_ack2;
    reg psw_ack3;
    reg psw_ack4;

    buffer inst1 (.out(out1), .in(in));
    buffer inst2 (.out(out2), .in(out1));

endmodule

```

- demo.upf:

```

set_design_top testbench

#####
## CREATE POWER DOMAINS ##
#####
create_power_domain Island_V1 -elements {inst1}
create_power_domain Island_V2 -elements {inst2}
create_power_domain Island_Vdd

#####
## CREATE SUPPLY NETS ##
#####
create_supply_net V1 -domain Island_V1
create_supply_net V1SW -domain Island_V1
create_supply_net V2 -domain Island_V2
create_supply_net Vdd -domain Island_Vdd
create_supply_net VSS -domain Island_V1
create_supply_net VSS -domain Island_V2 -reuse
create_supply_net VSS -domain Island_Vdd -reuse

```



```
#####
## CONNECT SUPPLY NETS ##
#####
connect_supply_net V1 -ports {V1_wire}
connect_supply_net V2 -ports {V2_wire}
connect_supply_net Vdd -ports {Vdd_wire}
connect_supply_net VSS -ports {Vss_wire}

#####
## DOMAIN SUPPLY NET ##
#####
set_domain_supply_net Island_V1 -primary_power_net V1SW
    -primary_ground_net VSS
set_domain_supply_net Island_V2 -primary_power_net V2
    -primary_ground_net VSS
set_domain_supply_net Island_Vdd -primary_power_net Vdd
    -primary_ground_net VSS

#####
## CREATE POWER SWITCH ##
#####
create_power_switch V1_header_switch -domain Island_V1
    -input_supply_port {V1 V1} -output_supply_port {V1SW V1SW}
    -control_port {ctrl_sig psw_ctrl} -control_port {ctrl_sig2
psw_ctrl2} -on_state {V1_ON V1 {!ctrl_sig}} -off_state {V1_OFF
{ctrl_sig}} -ack_port {ACK psw_ack ctrl_sig} -ack_port {ACK2
psw_ack2 !ctrl_sig2} -ack_port {ACK3 psw_ack3 ctrl_sig} -ack_port
{ACK4 psw_ack4} -ack_delay {ACK 10} -ack_delay {ACK2 20} -ack_delay
{ACK3 10} -ack_delay {ACK4 20}

#####
## CREATE PST ##
#####
add_port_state V1_header_switch/V1SW -state {SW_ON 1.2} -state {SW_OFF
0.0}
create_pst ls -supplies {V1_header_switch/V1SW}
add_pst_state ALL_ON -pst ls -state {SW_ON}
add_pst_state V1_HEADER_OFF -pst ls -state {SW_OFF}
```

- To compile demo.v and demo.upf:

```
vcs -sverilog demo.v -upf demo.upf -power=upfim -kdb
```

- To run a test.py with compiled design:

```
./test.py -dbdir simv.daidir
```

Public Function

Public Function contains the following section:

upf.handle_by_name(*pathname*, *relativeTo*)

Get a handle to an object with a specific name.

Parameters:

- **pathname(*str*)** – The string representing name of an object.
- **relativeTo(*handle*)** – An optional handle to the object from which the relative pathname is given.

Returns: The handle to the specified name or None if not found.

Return type: [BaseNamedSet](#)

Examples

```
>>> pd = upf.handle_by_name("testbench/Island_Vdd", None)
>>> print(pd.full_name() + " (" + pd.type() + ")")
testbench/Island_Vdd (npiUpfPowerDomain)
```

upf.get_top_scope()

Get a handle to the top power scope.

Parameters: None. –

Returns: The handle to the top power scope.

Return type: [PowerScopeHdl](#)

Examples

```
>>> scope = upf.get_top_scope()
>>> print(pd.full_name() + " (" + pd.type() + ")")
testbench (npiUpfPowerScope)
```

upf.get_power_domains()

Get a list of all power domains.

Parameters: None. –

Returns: The list of power domains.

Return type: list

Examples

```
>>> pd_list = upf.get_power_domains()
>>> print(*([pd.full_name() + " (" + pd.type() + ")" for pd in pd_list]),
      sep = "\n")
testbench/Island_V1 (npiUpfPowerDomain)
testbench/Island_V2 (npiUpfPowerDomain)
testbench/Island_Vdd (npiUpfPowerDomain)
```

upf.get_power_switches()

Get a list of all power switches.

Parameters: None. –

Returns: The list of power switches.

Return type: list

Examples

```
>>> ps_list = upf.get_power_switches()
>>> print(*([ps.full_name() + " (" + ps.type() + ")" for ps in ps_list]),
      sep = "\n")
testbench/V1_header_switch (npiUpfPowerSwitch)
```

upf.get_power_state_tables()

Get a list of all power state tables.

Parameters: None. –

Returns: The list of power state tables.

Return type: list

Examples

```
>>> pst_list = upf.get_power_state_tables()
>>> print(*([pst.full_name() + " (" + pst.type() + ")" for pst in
      pst_list]), sep = "\n")
testbench/ls (npiUpfPowerStateTable)
```